# LAPORAN PRAKTIKUM

# TEKNOLOGI IoT II

## MODUL 7: INTEGRASI BACKEND SERVER LOKAL (DOCKER & MOSQUITTO) DAN RULE ENGINE (NODE-RED)


**Disusun Oleh:**

**Salma Anggun Velyza (25782082)**

**TRI 3C**

**Dosen Pengajar:**

**Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.**

**Agus Ambarwari, S.Pd., M.Kom.**



**TEKNOLOGI REKAYASA INTERNET**

**JURUSAN TEKNOLOGI INFORMASI**

**POLITEKNIK NEGERI LAMPUNG**

**2025/2026**

## Tujuan Pembelajaran (Bagian Awal)

- Mahasiswa mampu menjelaskan prinsip dasar arsitektur *Edge/Fog Computing*, keunggulan server lokal dibanding broker publik, serta konsep kontainerisasi menggunakan Docker.
- Mahasiswa mampu menyusun dokumen orkestrasi multi-kontainer docker-compose.yml untuk menjalankan layanan Eclipse Mosquitto dan Node-RED secara terpadu.
- Mahasiswa mampu menyusun berkas konfigurasi mosquitto.conf untuk mendukung protokol komunikasi MQTT (port 1883) dan WebSockets (port 9001).

### Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

#### *1. Langkah Kerja / Implementasi:*

1. Membuat direktori kerja baru bernama iot-server di dalam penyimpanan lokal (misalnya pada direktori D:/iot-server).
2. Membuat struktur folder sub-direktori iot-server/mosquitto/config/.
3. Membuat dan mengisi berkas konfigurasi **mosquitto.conf** di dalam folder tersebut dengan parameter berikut:

   **listener 1883**
   **protocol mqtt**
   **listener 9001**
   **protocol websockets**
   **allow_anonymous true**
   **persistence true**
   **persistence_location /mosquitto/data/**
   **log_dest file /mosquitto/log/mosquitto.log**
   **log_dest stdout**

4. Membuat berkas orkestrasi **docker-compose.yml** di dalam direktori utama iot-server untuk menjalankan layanan kontainer Mosquitto MQTT Broker dan Node-RED secara bersamaan.
5. Menjalankan terminal PowerShell di direktori D:\iot-server dan mengeksekusi perintah untuk mengunduh *image* serta menghidupkan kontainer di latar belakang:

   **docker compose up -d**

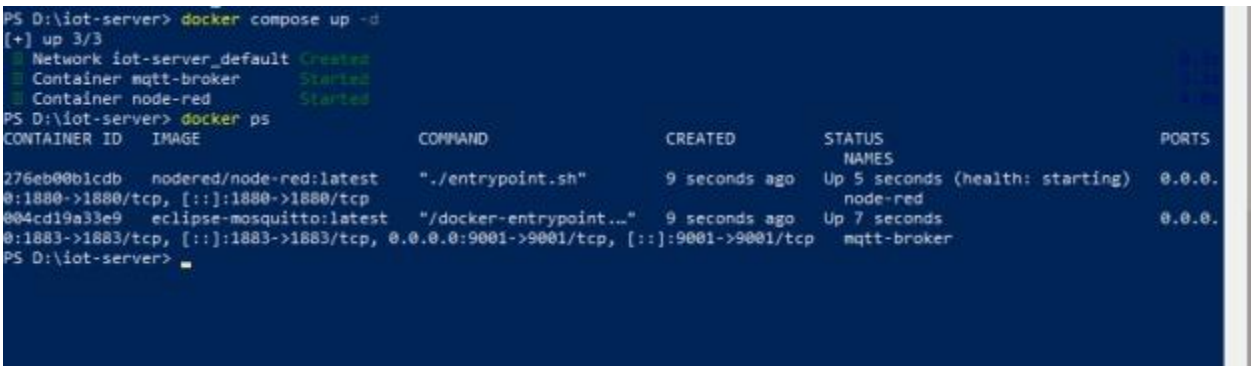
6. **Catatan Kendali & Penanganan Masalah (*Troubleshooting*):**

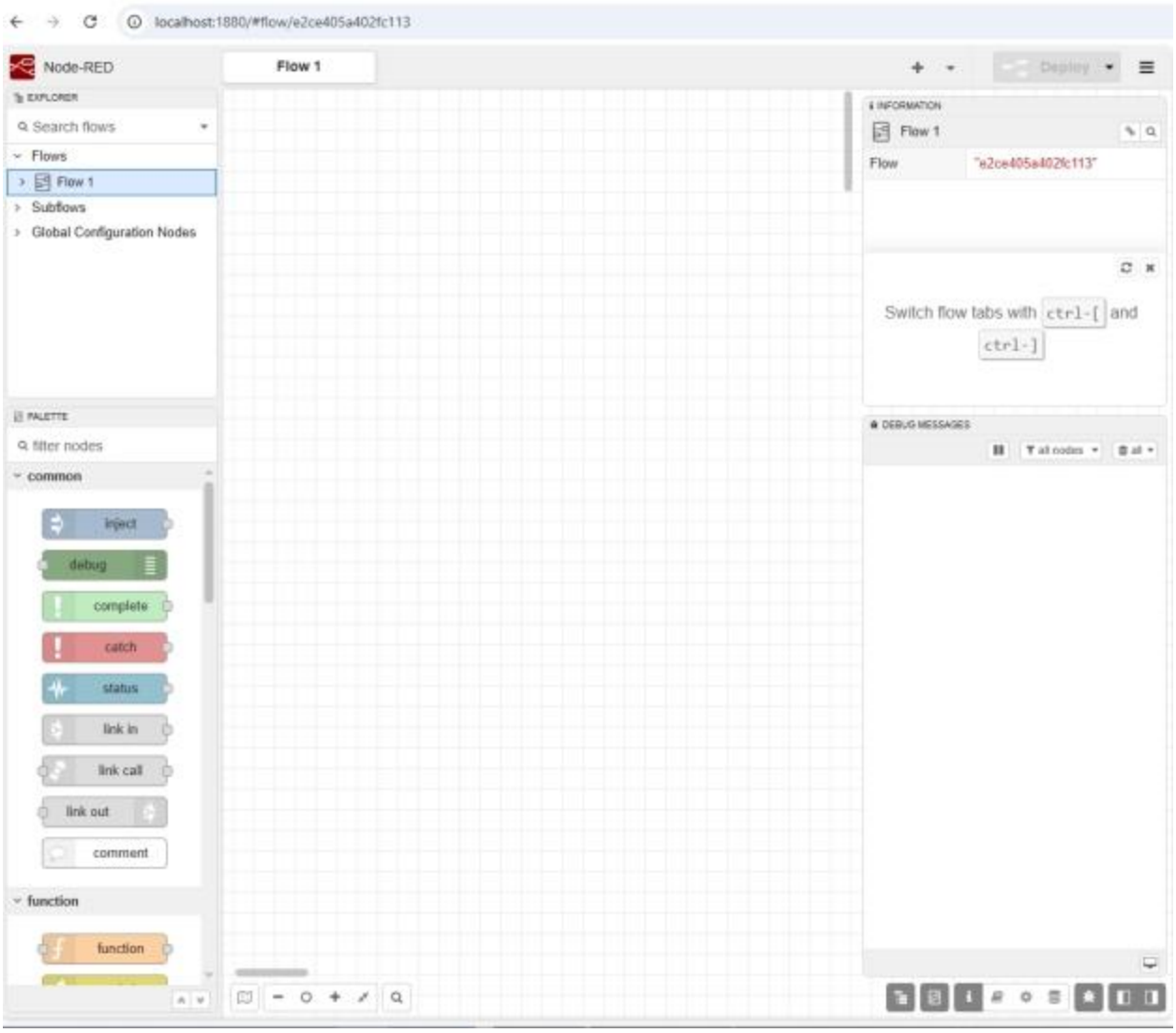
- Sempat terjadi kendala di mana kontainer mqtt-broker mengalami *restart* berulang akibat penyesuaian direktori *volume* lokal Windows.
- Masalah diselesaikan dengan membersihkan ulang direktori *volume* data Mosquitto, memperbarui struktur berkas konfigurasi mosquitto.conf menggunakan editor teks standar, dan menjalankan ulang perintah orkestrasi:

**docker compose down**
**docker compose up -d**

7. Melakukan verifikasi akhir menggunakan perintah docker ps dan memastikan status layanan mqtt-broker serta node-red telah berjalan stabil dengan status **Up**.
8. Mengakses antarmuka visual Node-RED melalui peramban web pada alamat URL **http://localhost:1880**.

**Hasil & Bukti Tangkapan Layar (Screenshot):**

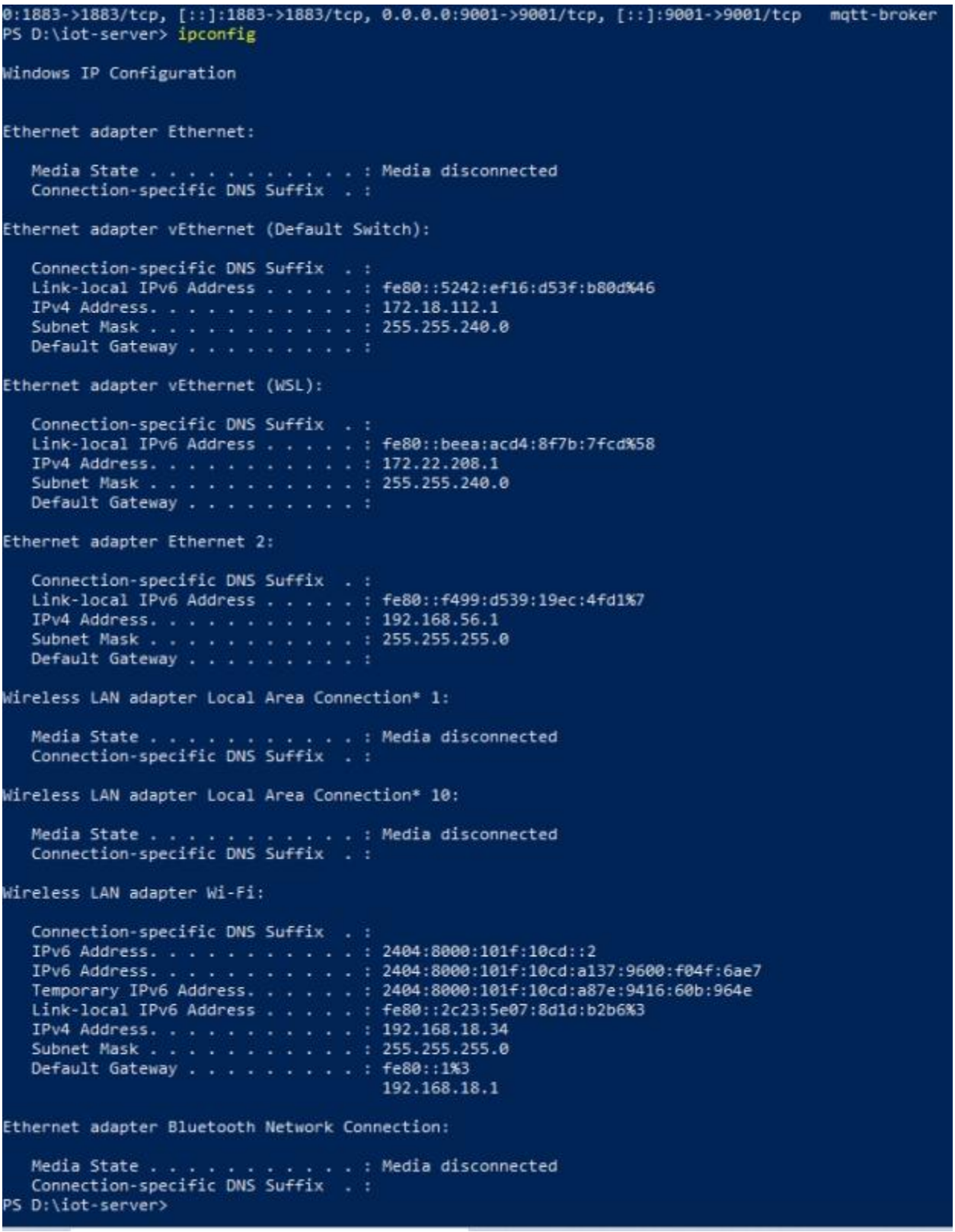


**Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal**

1. Konfigurasi Rangkaian Perangkat Keras:
    - Menyiapkan mikrokontroler NodeMCU ESP8266 dan papan *breadboard*.
    - Menghubungkan sensor DHT11/DHT22: Pin VCC ke 3V3, Pin GND ke GND, dan Pin Data ke Pin D4 (GPIO 2).
    - Menghubungkan aktuator LED Indikator: Anoda (kaki panjang) ke Pin D1 (GPIO 5) dan Katoda (kaki pendek) melalui resistor 220  ke GND.
2. Identifikasi Alamat IP Lokal Laptop:
    - Mengecek alamat IPv4 laptop di jaringan Wi-Fi lokal menggunakan perintah ipconfig pada Command Prompt/PowerShell.
3. Pengaturan Firewall Windows:
    - Memastikan aturan masuk (*Inbound Rules*) pada *Windows Defender Firewall* diaktifkan untuk port 1883 agar paket data dari NodeMCU tidak diblokir oleh sistem.
4. Penyusunan Program Arduino IDE:
    - Menulis dan menyesuaikan sketsa kode program pada Arduino IDE dengan memasukkan kredensial Wi-Fi lokal serta alamat IP laptop sebagai mqttServer.
    - Mengatur fungsi penanganan pesan masuk (onMessageCallback) untuk mengurai data JSON kendali LED ({"led": 1} atau {"led": 0}).
    - Mengatur fungsi pengiriman telemetri berkala non-blokir setiap 4 detik ke topik kampus/labiot/telemetri.
5. Unggah Program & Verifikasi:
    - Melakukan *Upload* program ke board NodeMCU ESP8266.
    - Membuka *Serial Monitor* (115200 baud) untuk memastikan mikrokontroler berhasil terhubung ke jaringan Wi-Fi serta sukses melakukan koneksi MQTT ke broker lokal (Terhubung!).

**Hasil & Bukti Tangkapan Layar (Screenshot):**



```
0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp   mqtt-broker
PS D:\iot-server> ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Ethernet adapter vEthernet (Default Switch):

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::5242:ef16:d53f:b80d%46
   IPv4 Address. . . . . . . . . . . : 172.18.112.1
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . :

Ethernet adapter vEthernet (WSL):

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::beea:acd4:8f7b:7fcd%58
   IPv4 Address. . . . . . . . . . . : 172.22.208.1
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . :

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::f499:d539:19ec:4fd1%7
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 10:

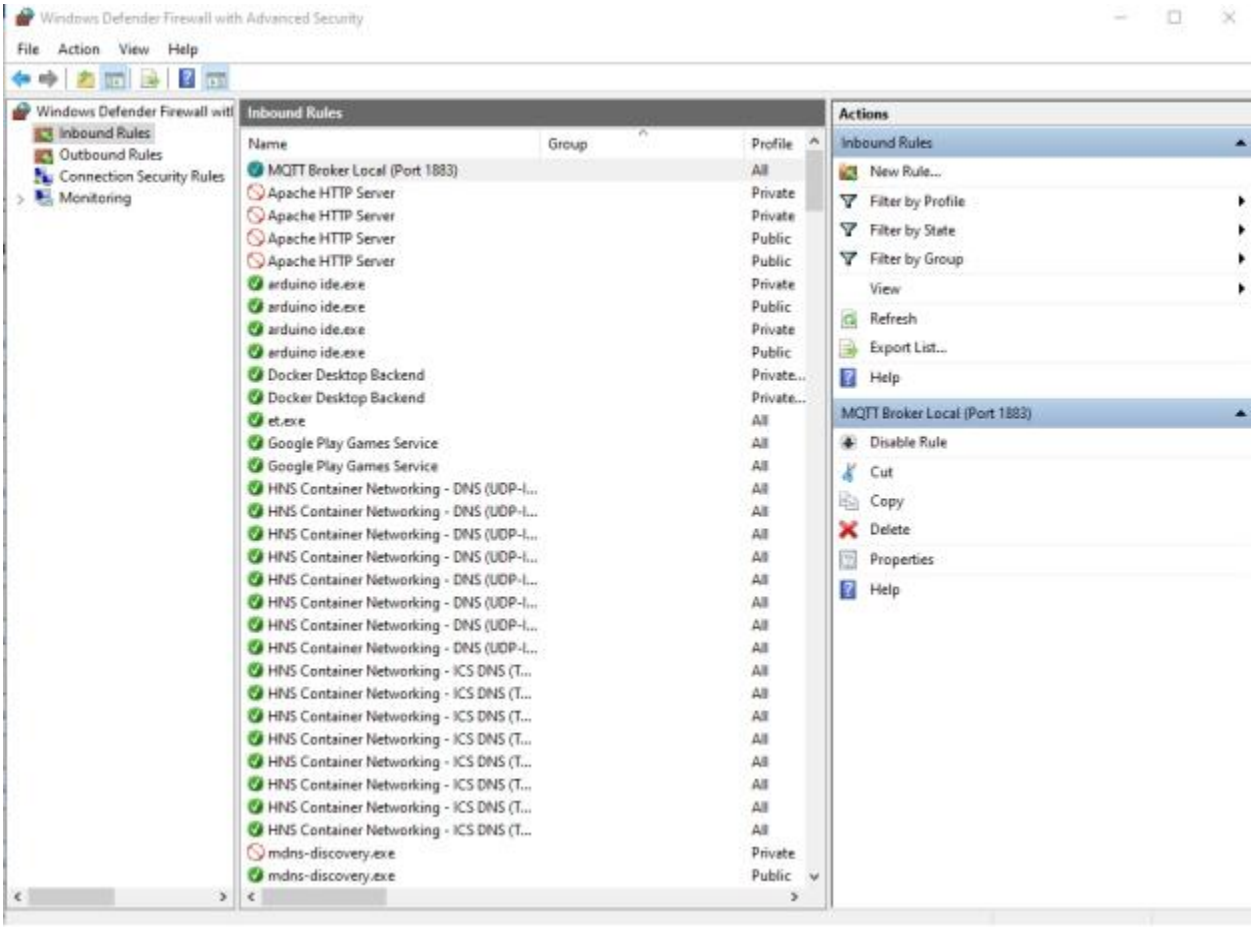
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

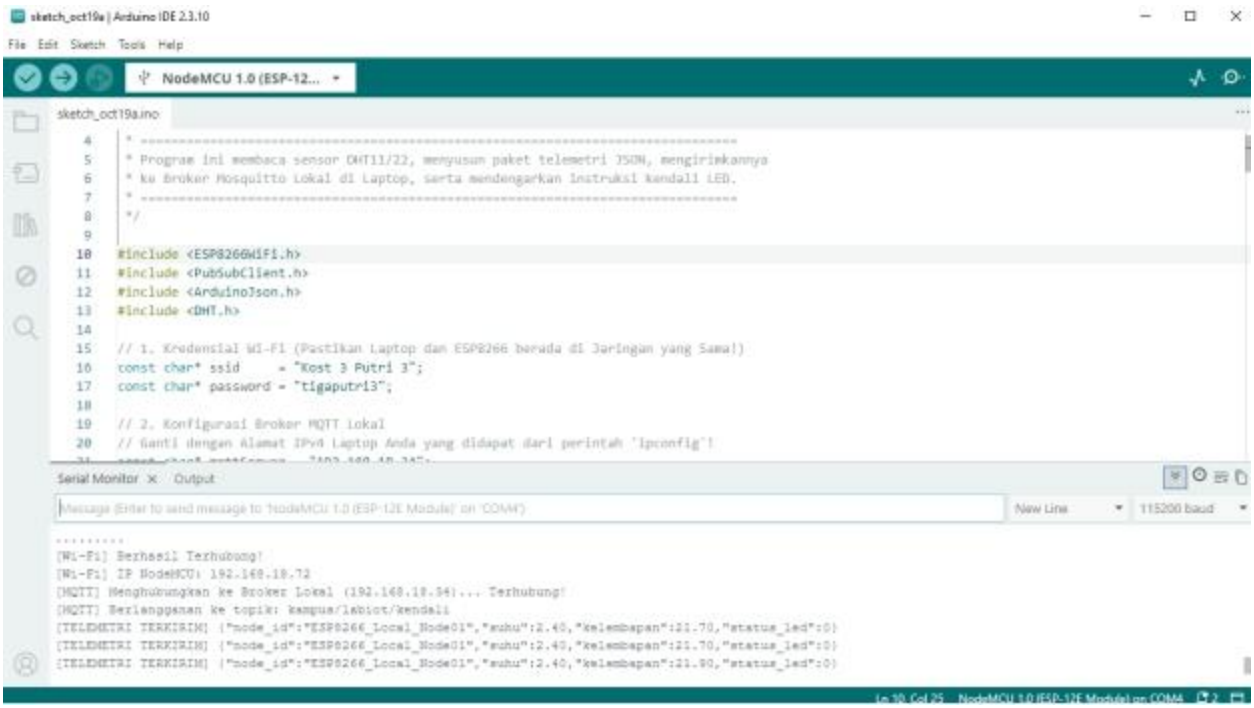
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2404:8000:101f:10cd::2
   IPv6 Address. . . . . . . . . . . : 2404:8000:101f:10cd:a137:9600:f04f:6ae7
   Temporary IPv6 Address. . . . . . : 2404:8000:101f:10cd:a87e:9416:60b:964e
   Link-local IPv6 Address . . . . . : fe80::2c23:5e07:8d1d:b2b6%3
   IPv4 Address. . . . . . . . . . . : 192.168.18.34
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::1%3
                                       192.168.18.1

Ethernet adapter Bluetooth Network Connection:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
PS D:\iot-server>
```


Windows Defender Firewall with Advanced Security
File Action View Help
Windows Defender Firewall with
Inbound Rules
Outbound Rules
Connection Security Rules
Monitoring
Inbound Rules
Name
Group
Profile
MQTT Broker Local (Port 1883) All
Apache HTTP Server Private
Apache HTTP Server Private
Apache HTTP Server Public
Apache HTTP Server Public
arduino ide.exe Private
arduino ide.exe Public
arduino ide.exe Private
arduino ide.exe Public
Docker Desktop Backend Private...
Docker Desktop Backend Private...
et.exe All
Google Play Games Service All
Google Play Games Service All
HNS Container Networking - DNS (UDP-I... All
HNS Container Networking - DNS (UDP-I... All
HNS Container Networking - DNS (UDP-I... All
HNS Container Networking - DNS (UDP-I... All
HNS Container Networking - DNS (UDP-I... All
HNS Container Networking - DNS (UDP-I... All
HNS Container Networking - DNS (UDP-I... All
HNS Container Networking - DNS (UDP-I... All
HNS Container Networking - ICS DNS (T... All
HNS Container Networking - ICS DNS (T... All
HNS Container Networking - ICS DNS (T... All
HNS Container Networking - ICS DNS (T... All
HNS Container Networking - ICS DNS (T... All
HNS Container Networking - ICS DNS (T... All
HNS Container Networking - ICS DNS (T... All
mdns-discovery.exe Private
mdns-discovery.exe Public
Actions
Inbound Rules
New Rule...
Filter by Profile
Filter by State
Filter by Group
View
Refresh
Export List...
Help
MQTT Broker Local (Port 1883)
Disable Rule
Cut
Copy
Delete
Properties
Help


sketch_oct19a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct19a.ino
* Program ini membaca sensor DHT11/22, menyusun paket telemetri JSON, mengirimkannya
* ke Broker Mosquitto Lokal di Laptop, serta mendengarkan instruksi kendali LED.
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>
// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";
// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
Serial Monitor
Output
New Line
115200 baud
[Wi-Fi] Berhasil Terhubung!
[Wi-Fi] IP NodeMCU: 192.168.18.72
[MQTT] Menghubungkan ke Broker Lokal (192.168.18.34)... Terhubung!
[MQTT] Berlangganan ke topik: kampus/lab1ot/kendali
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":2.40,"kelembapan":21.70,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":2.40,"kelembapan":21.70,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":2.40,"kelembapan":21.90,"status_led":0}

**Hasil Pengujian Sinyal dan Koneksi:**

- Berdasarkan pengujian menggunakan *Serial Monitor* Arduino IDE (kecepatan 115200 baud), mikrokontroler NodeMCU ESP8266 berhasil terhubung ke jaringan Wi-Fi lokal (*Kost 3 Putri 3*) dan mendapatkan alamat IP lokal **192.168.18.72**.
- NodeMCU juga sukses melakukan jabat tangan koneksi (*handshake*) dan terhubung secara stabil ke *Eclipse Mosquitto MQTT Broker* lokal yang berjalan di alamat IP laptop **192.168.18.34** pada port standar 1883.

**Contoh Isi Log / Payload Telemetri yang Berhasil Dikirim:**

[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":2.40,"kelembapan":21.70,"status_led":0}

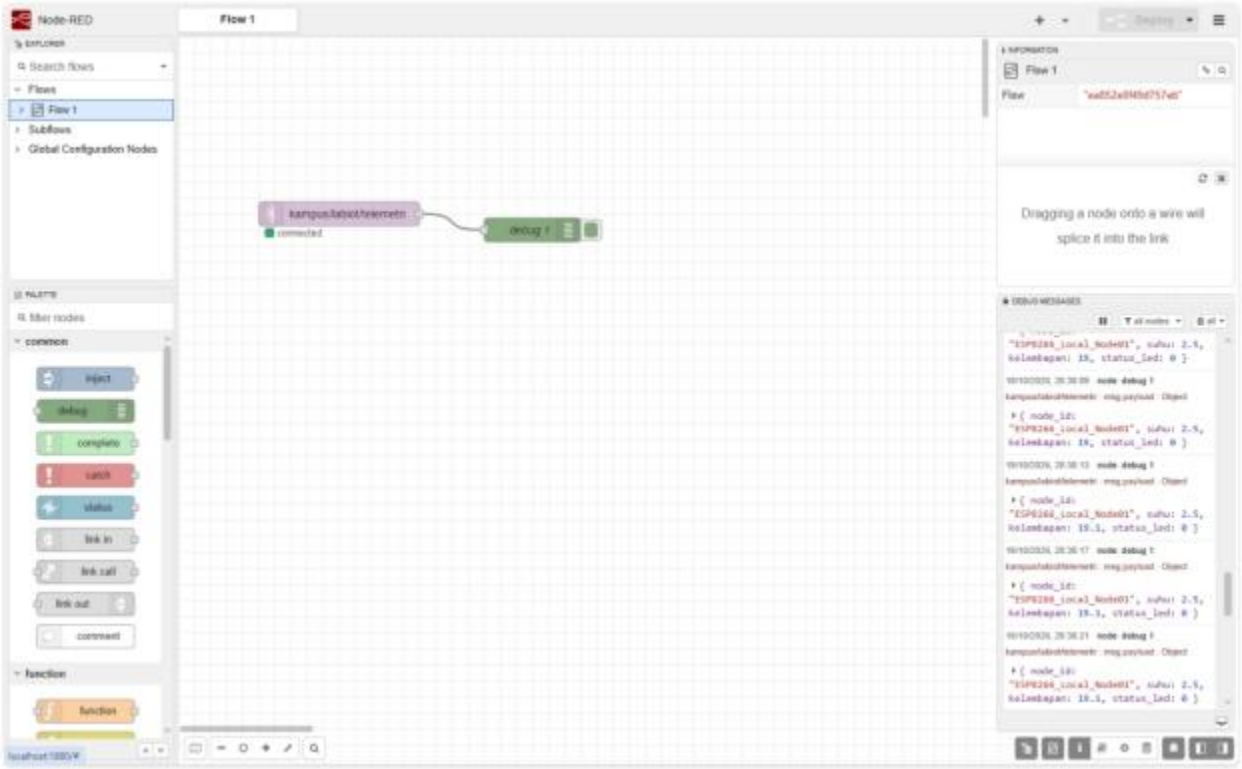
**Analisis Pembahasan:**

- Pada praktikum ini, komunikasi data telemetri berhasil berjalan secara lokal menggunakan protokol MQTT. Penggunaan IP laptop sebagai mqttServer memastikan bahwa paket data sensor dari modul DHT11 pada NodeMCU dikirim langsung ke server Mosquitto di dalam jaringan *Local Area Network (LAN)* tanpa harus melalui internet publik (Broker Publik seperti HiveMQ).
- Status awal status_led: 0 menunjukkan bahwa aktuator LED fisik pada pin D1 berada dalam kondisi padam (*OFF*) sebelum menerima instruksi kendali otomatis lanjutan.

**Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED**

**Tujuan & Konfigurasi Alur Kerja (*Flow*):**

- Menangkap, mengurai, dan memantau aliran data telemetri yang dikirimkan oleh mikrokontroler NodeMCU ESP8266 secara *real-time* menggunakan antarmuka visual Node-RED.
- Menambahkan node mqtt in pada *Palette* kategori *network* ke kanvas kerja, lalu mengatur parameternya:
    - Server: Dikonfigurasi menggunakan nama servis kontainer Docker yaitu mqtt (pada port 1883) agar terhubung langsung dengan *Mosquitto Local*.
    - Topic: kampus/labiot/telemetri
    - Output: Diubah dari *auto-detect* menjadi a parsed JSON object agar format teks JSON otomatis terurai menjadi objek JavaScript hierarkis.
- Menambahkan node debug dari kategori *common* dan menghubungkannya (*wiring*) dengan node mqtt in.
- Mengaktifkan perubahan dengan menekan tombol Deploy, di mana indikator di bawah node mqtt in berubah menjadi hijau dengan status connected.



**Contoh Debug Messages yang Masuk secara *Real-Time*:**

```
19/10/2026, 20:38:05 [node: debug 1] kampus/labiot/telemetri : msg.payload : Object
{
  node_id: "ESP8266_Local_Node01",
  suhu: 2.5,
  kelembapan: 19,
  status_led: 0
}
```

## Analisis & Pembahasan:

- Penggunaan tipe *Output* sebagai objek JSON terurai sangat mempermudah pemrosesan data selanjutnya tanpa memerlukan fungsi *parser* manual.
- Data telemetri sensor dari NodeMCU terbukti sukses diterima oleh server Node-RED lokal setiap beberapa detik secara stabil tanpa hambatan latensi jaringan internet publik.

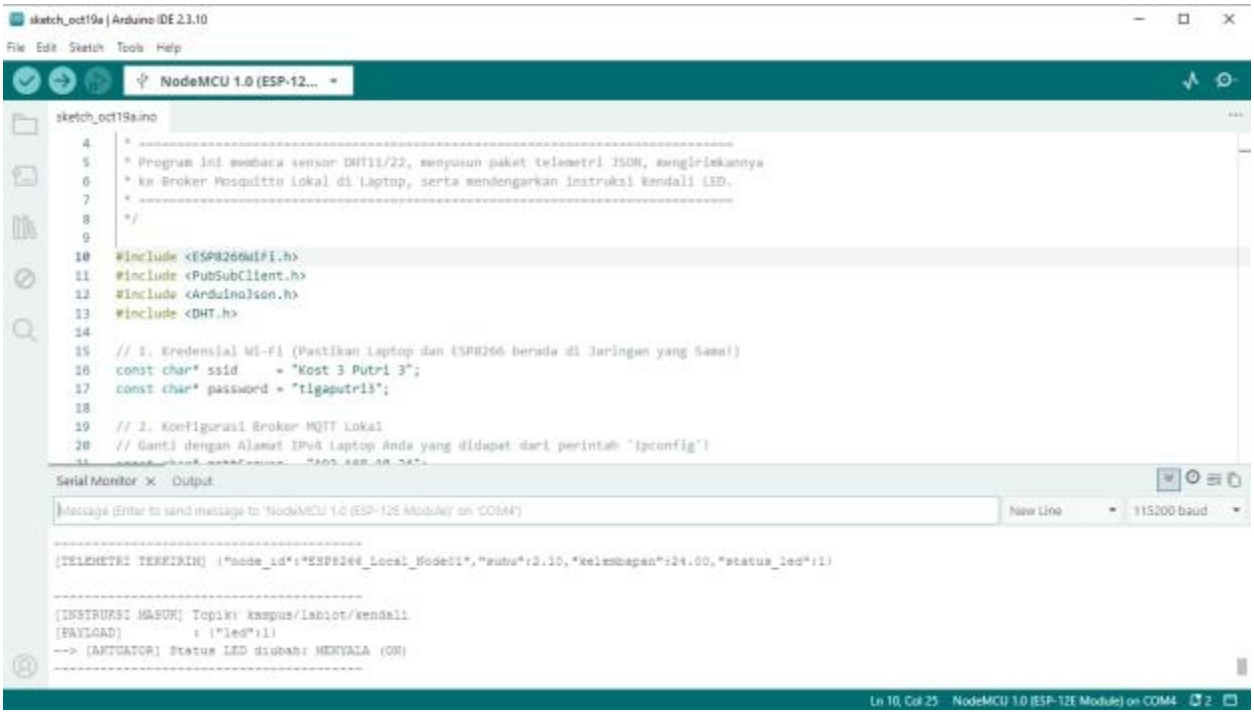
**Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup**

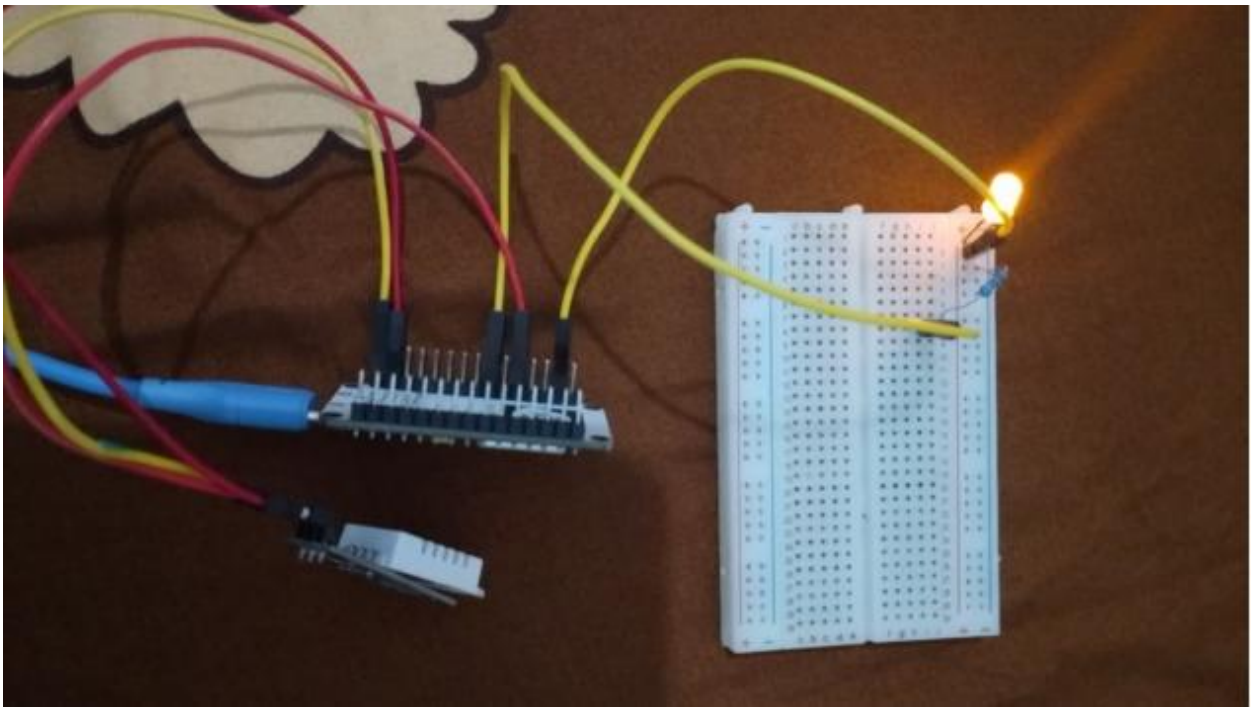
**Tujuan & Konfigurasi Alur Kerja (*Flow*):**

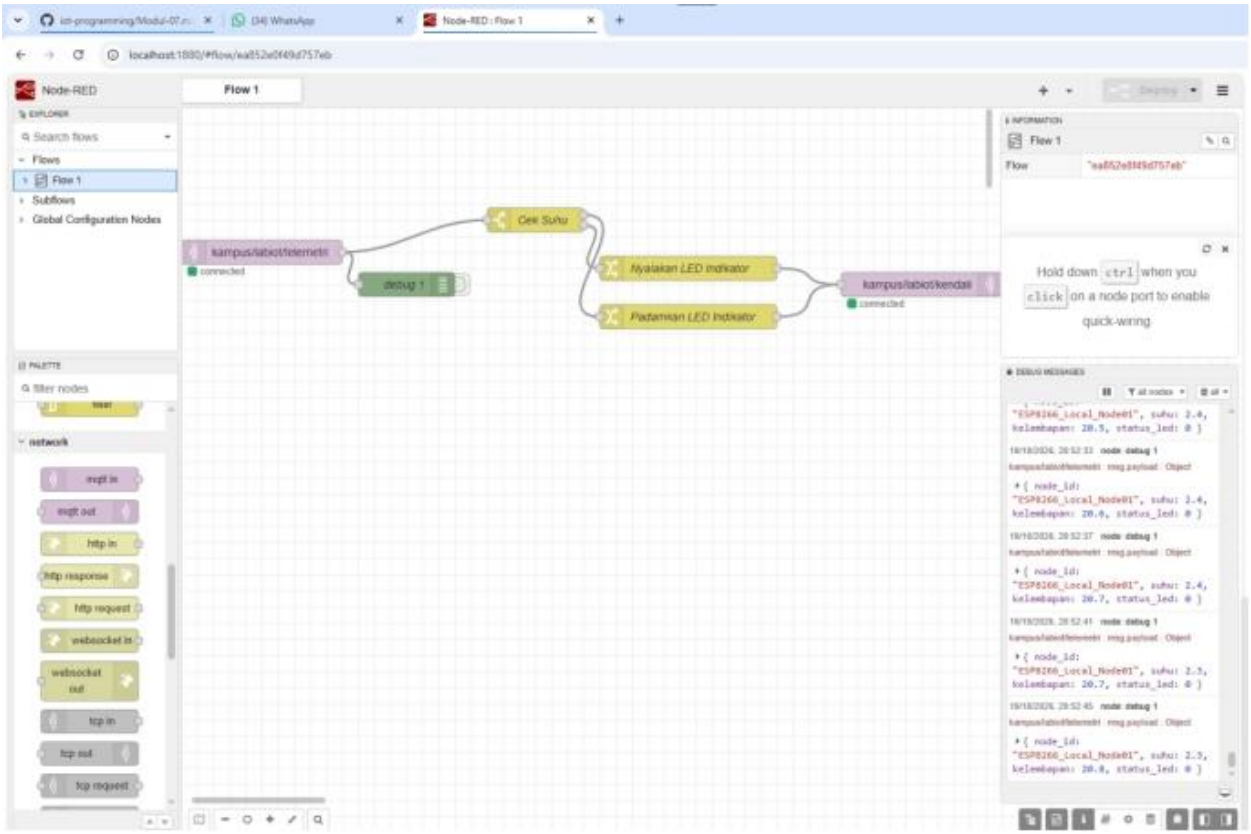
- Membangun sistem kendali otonom (*Closed-Loop Rule Engine*) berbasis ambang batas suhu (*threshold control*):
    - Kondisi Panas (> 31°C): Node-RED memicu aturan nomor 1 dan mengubah muatan pesan (*payload*) menjadi format JSON {"led": 1} untuk menyalakan indikator fisik LED di pin D1.
    - Kondisi Aman (≤ 31°C): Node-RED memicu aturan nomor 2 dan mengubah muatan pesan (*payload*) menjadi format JSON {"led": 0} untuk memadamkan LED.
- Komponen Node yang Digunakan:
    - Node switch (diberi properti msg.payload.suhu dengan dua aturan percabangan: > 31 dan <= 31).
    - Node change 1 ("*Nyalakan LED Indikator*") untuk mengatur *payload* menjadi {"led": 1}.
    - Node change 2 ("*Padamkan LED Indikator*") untuk mengatur *payload* menjadi {"led": 0}.
    - Node mqtt out yang terhubung ke topik kendali kampus/labiot/kendali pada *Mosquitto Local*.

**Bukti Hasil Pengujian:**

- Saat suhu ruangan terbaca di bawah batas normal (≤ 31°C), sistem mengirimkan instruksi pemadaman ({"led": 0}) dan LED indikator tetap padam.
- Ketika sensor mendeteksi kenaikan suhu melewati 31°C, Node-RED secara otomatis mengirimkan paket kendali ({"led": 1}) ke NodeMCU, dan LED indikator langsung menyala secara *real-time*.


sketch_oct19a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct19a.ino
* Program ini membaca sensor DHT11/22, menyusun paket telemetri JSON, mengirimkannya
* ke Broker Mosquitto Lokal di Laptop, serta mendengarkan instruksi kendali LED.
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>
// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";
// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
Serial Monitor
Output
New Line
115200 baud
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.10,"kelembapan":24.00,"status_led":1}
[INSTRUKSI MASUK] Topik: kampus/lab1ot/kendali
[PAYLOAD] : {"led":1}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)
Ln 10, Col 25 NodeMCU 1.0 (ESP-12E Module) on COM4




Node-RED
Flow 1
Search flows
Flows
Flow 1
Subflows
Global Configuration Nodes
kampus/laboit/telemetri
connected
debug 1
Cek Suhu
Nyalakan LED indikator
Padamkan LED indikator
kampus/laboit/kendali
connected
Hold down ctrl when you click on a node port to enable quick-wiring
filter nodes
network
mqtt in
mqtt out
http in
http response
http request
websocket in
websocket out
tcp in
tcp out
tcp request

## LATIHAN

### Latihan 1: Analisis Jaringan Internal Docker & Service Discovery

1. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?
    - Di dalam ekosistem Docker, setiap kontainer berjalan sebagai lingkungan sistem terisolasi (*isolated container environment*) yang memiliki *network namespace* dan *loopback interface* (127.0.0.1 atau localhost miliknya sendiri).
    - Jika Node-RED diset terhubung ke localhost:1883, ia akan mencari broker Mosquitto di dalam kontainer Node-RED itu sendiri, bukan di dalam kontainer mqtt-broker. Karena kontainer Node-RED tidak menjalankan layanan Mosquitto, koneksi akan ditolak dan memunculkan *Connection Refused*.
2. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!
    - Saat kita menggunakan *Docker Compose*, secara otomatis Docker membuat jaringan kustom bertipe *bridge network* khusus untuk layanan-layanan yang didefinisikan bersama.
    - Di dalam jaringan internal Docker ini, terdapat fitur Docker Embedded DNS Server yang secara otomatis menerjemahkan nama layanan (*service name*) yang didefinisikan pada docker-compose.yml (seperti mqtt) menjadi alamat IP internal kontainer yang bersangkutan. Oleh karena itu, kontainer node-red dapat langsung mengenali dan berkomunikasi dengan kontainer mqtt-broker menggunakan nama mqtt tanpa harus mengetahui alamat IP aslinya.

### Latihan 2: Penanganan Masalah Konektivitas Jaringan & Subnet Firewall

1. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?
    - NodeMCU berada pada kelompok jaringan (*subnet*) yang berbeda (misalnya dari jaringan *hotspot* 192.168.43.x) dibandingkan dengan laptop server yang saat ini aktif menggunakan jaringan Wi-Fi dengan alamat IP 192.168.18.34.
    - Perangkat yang berada pada subnet berbeda tidak dapat saling berkomunikasi secara langsung menggunakan protokol lapisan tautan data lokal (*ARP/Broadcast*) tanpa adanya perangkat perute (*router*) atau aturan *routing* khusus yang menghubungkan kedua kelompok jaringan tersebut. Akibatnya, paket MQTT dari NodeMCU tidak akan pernah sampai ke alamat IP laptop (192.168.18.34).
2. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?
    - Menyamakan Sumber Jaringan: Pastikan kedua perangkat (laptop dan NodeMCU) terhubung ke jaringan Wi-Fi atau sumber *hotspot* yang persis sama. Konfigurasi SSID dan *password* pada program Arduino IDE harus disesuaikan agar NodeMCU terhubung ke jaringan yang sama dengan laptop (sehingga mendapatkan rentang alamat IP yang satu subnet dengan 192.168.18.xx).

- Konfigurasi Firewall (Pendukung): Jika subnet jaringan sudah sama tetapi koneksi masih terhalang, pastikan aturan masuk (*Inbound Rules*) pada *Windows Defender Firewall* di laptop sudah diatur untuk mengizinkan akses koneksi TCP pada port 1883.

## Latihan 3: Implementasi Filter Histeresis pada Rule Engine

Pada sistem kendali dua posisi (*Bang-Bang*), untuk mencegah terjadinya kedip-kedip cepat (*flickering / hunting*) pada lampu LED saat suhu berfluktuasi tipis di sekitar titik batas, kita menggunakan **Function Node** yang memuat logika histeresis.

**kode skrip yang dimasukkan ke dalam *Function Node* tersebut:**

```
let statusLED = context.get('statusLED') || 0;

let suhu = msg.payload.suhu;

if (suhu > 31.5) {

   statusLED = 1;

} else if (suhu < 30.0) {

   statusLED = 0;

}

context.set('statusLED', statusLED);

msg.payload = { "led": statusLED };

return msg;
```

## Latihan 4: Analisis Mekanisme Volume Mount & Persistensi Data

1. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?
    - Seluruh alur kanvas visual (*flow*), riwayat data, *nodes* yang telah disusun, serta konfigurasi server yang tersimpan di dalam kontainer Node-RED akan hilang total (terhapus).
    - Ketika kontainer dijalankan ulang (docker compose up -d), Node-RED akan kembali seperti dalam kondisi baru (*fresh install*) yang kosong tanpa ada satupun *flow* latihan yang sebelumnya sudah dibuat. Hal ini terjadi karena penyimpanan data di dalam kontainer bersifat sementara (*ephemeral*), sehingga fitur *Volume Mount* diperlukan agar data tetap tersimpan secara permanen di direktori fisik laptop (*host*).
2. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

    - Bind Mount:

        - Mengaitkan (*mount*) direktori atau berkas tertentu yang ada secara langsung di dalam sistem operasi laptop induk (*host*) ke dalam kontainer.
        - Pengguna memiliki kontrol penuh terhadap lokasi penyimpanan di komputer (misalnya ./nodered:/data), sehingga berkas-berkas di dalamnya sangat mudah diakses, disunting, dicadangkan, atau dipindahkan langsung melalui *File Explorer* laptop.
    - Named Volume:
        - Dikelola sepenuhnya oleh Docker di dalam area penyimpanan internal Docker (biasanya terletak di dalam direktori sistem tersembunyi pada OS *host*).
        - Volume diberi nama khusus oleh pengguna (misalnya my-data:/data), dan Docker yang mengatur alokasi ruang penyimpanannya. *Named volume* lebih direkomendasikan untuk produksi murni karena terisolasi dari manipulasi berkas luar yang tidak sengaja oleh pengguna.

**TUGAS MODUL 7:**

Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

**Langkah-Langkah Pengerjaan**

- Pembuatan Kredensial: Mengeksekusi perintah Docker untuk membuat berkas password.txt terenkripsi dengan akun admin.
- Konfigurasi Mosquitto: Mengubah parameter allow_anonymous false dan menambahkan baris password_file di dalam mosquitto.conf, lalu merestart kontainer mqtt-broker.
- Analisis Kegagalan (rc=5): Menjelaskan hasil pengamatan ketika broker menolak koneksi karena belum adanya kredensial (terlihat pada Serial Monitor ESP8266 dan status *disconnected* di Node-RED).
- Implementasi Perbaikan:

    - Menambahkan parameter "admin", "rahasia123" pada fungsi client.connect() di sketsa Arduino IDE.
    - Mengisi tab *Security* dengan *Username* dan *Password* pada pengaturan server MQTT di Node-RED.

Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

- Membuat berkas kredensial terenkripsi di dalam kontainer menggunakan utilitas Mosquitto:

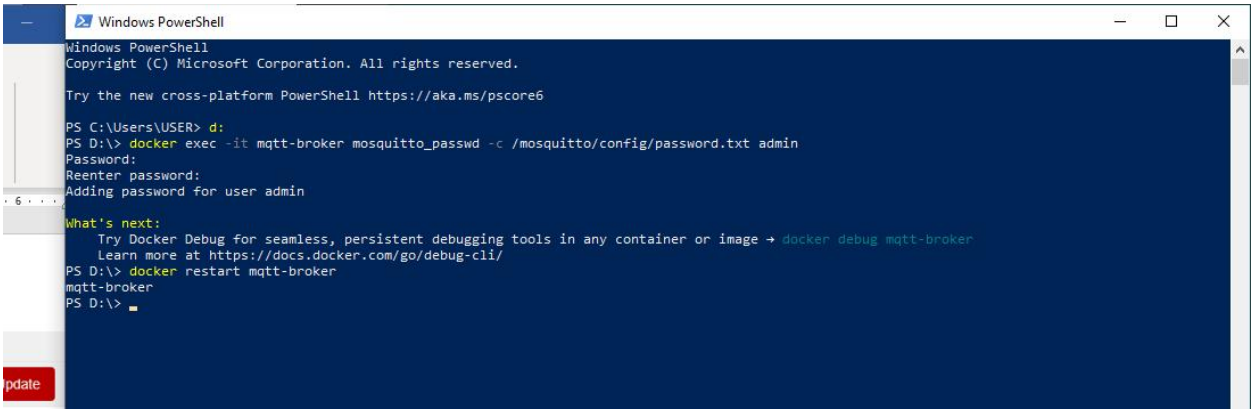
  docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin

- Mengubah berkas konfigurasi mosquitto.conf dengan menonaktifkan akses anonim dan mendaftarkan direktori berkas sandi:

  allow_anonymous false
  password_file /mosquitto/config/password.txt

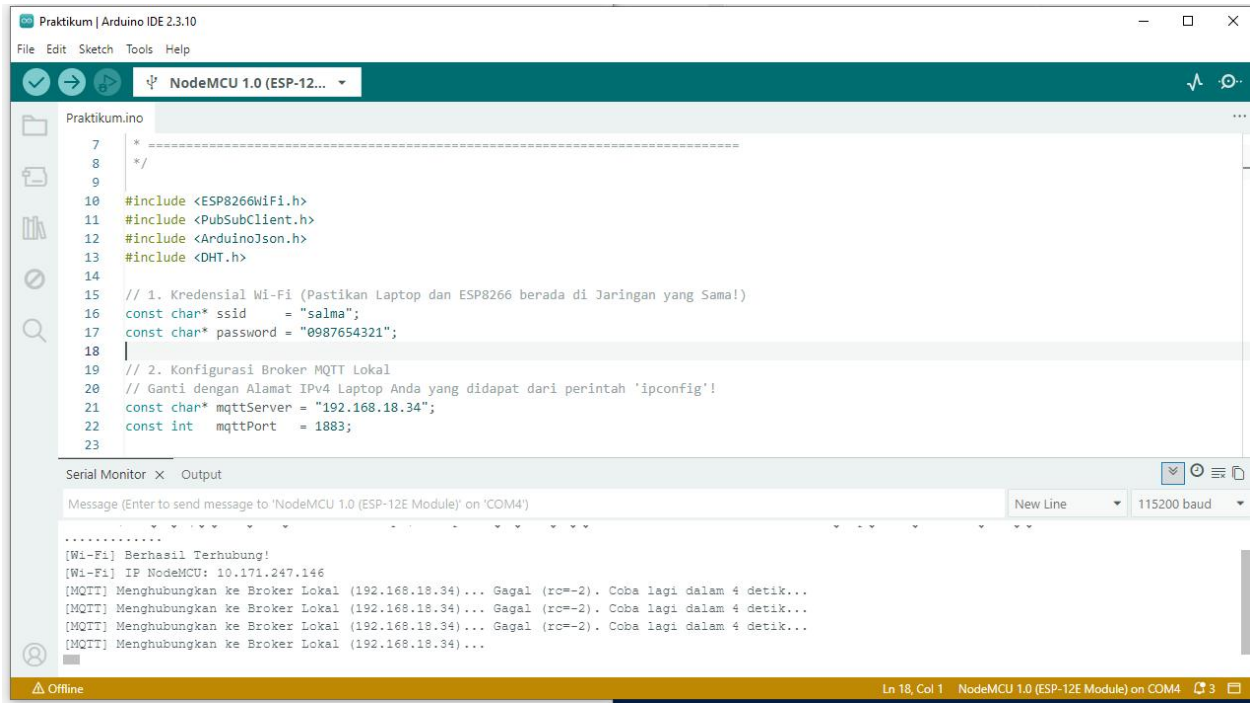
- Merestart layanan kontainer Mosquitto untuk menerapkan kebijakan keamanan baru:

  docker restart mqtt-broker



- **Dampak pada NodeMCU ESP8266 (Arduino IDE):**

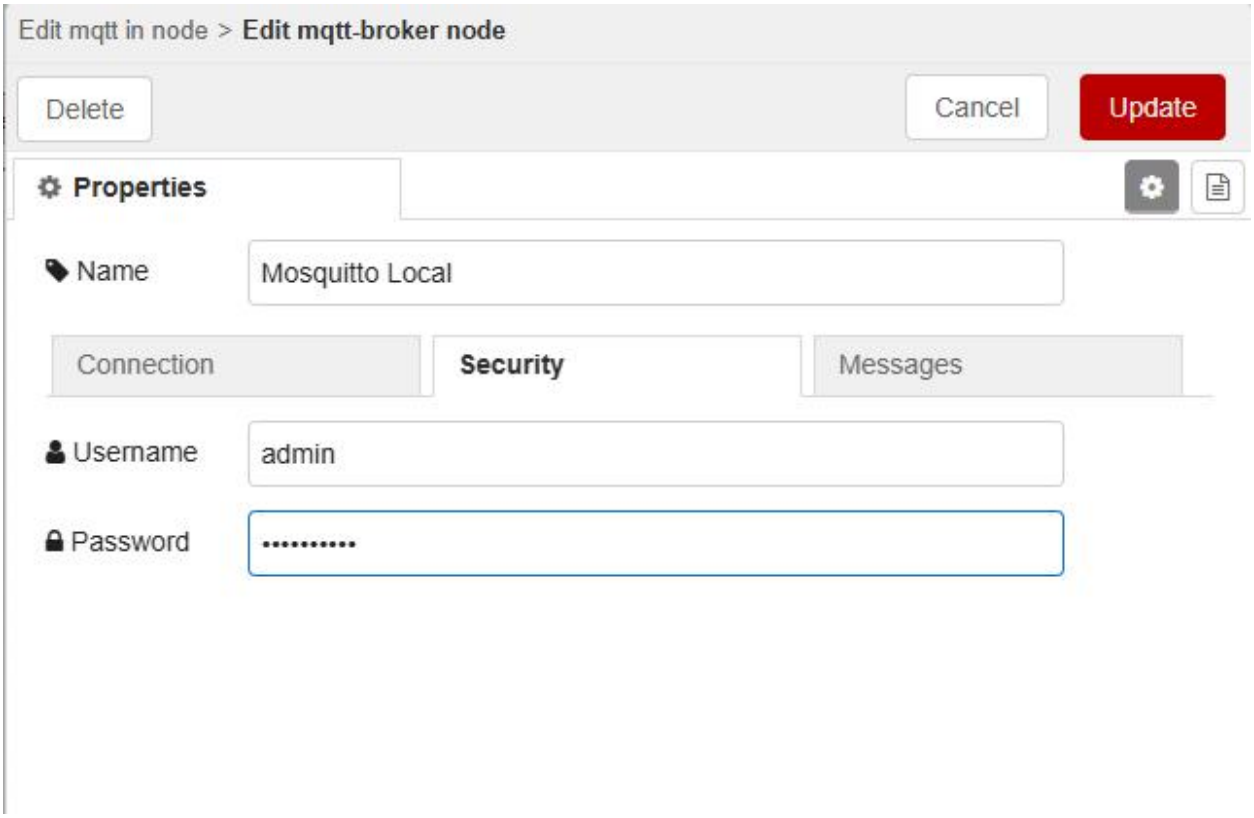
  Begitu fitur autentikasi diaktifkan, mikrokontroler yang belum diberi kredensial langsung ditolak oleh server. Pada Serial Monitor, muncul pesan kegagalan berulang:




Dampak pada Antarmuka Node-RED: Di browser, status node MQTT pada kanvas otomatis berubah menjadi warna merah (disconnected / connection refused karena belum memasukkan *username* dan *password*).

- **Memperbarui Program di Arduino IDE:**

- memodifikasi fungsi koneksi MQTT pada sketsa program Arduino dengan menyertakan parameter kredensial admin dan rahasia123 di dalam fungsi client.connect():
- Memperbarui Pengaturan Keamanan di Node-RED: Kamu mengeklik dua kali node MQTT di Node-RED, membuka tab Security, lalu memasukkan:
  - Username: admin
  - Password: rahasia123 Setelah itu mengeklik *Update* dan menekan tombol merah Deploy.

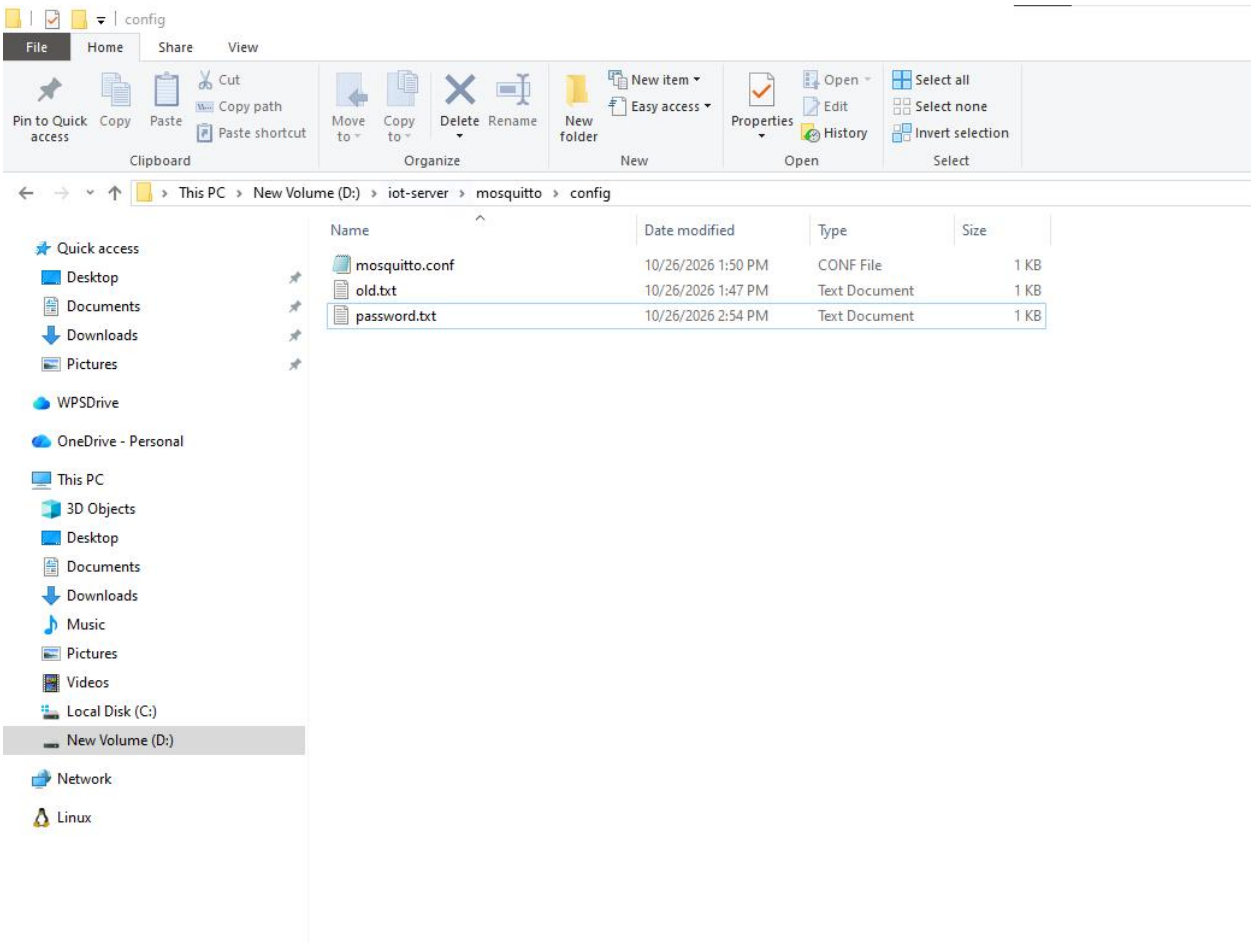


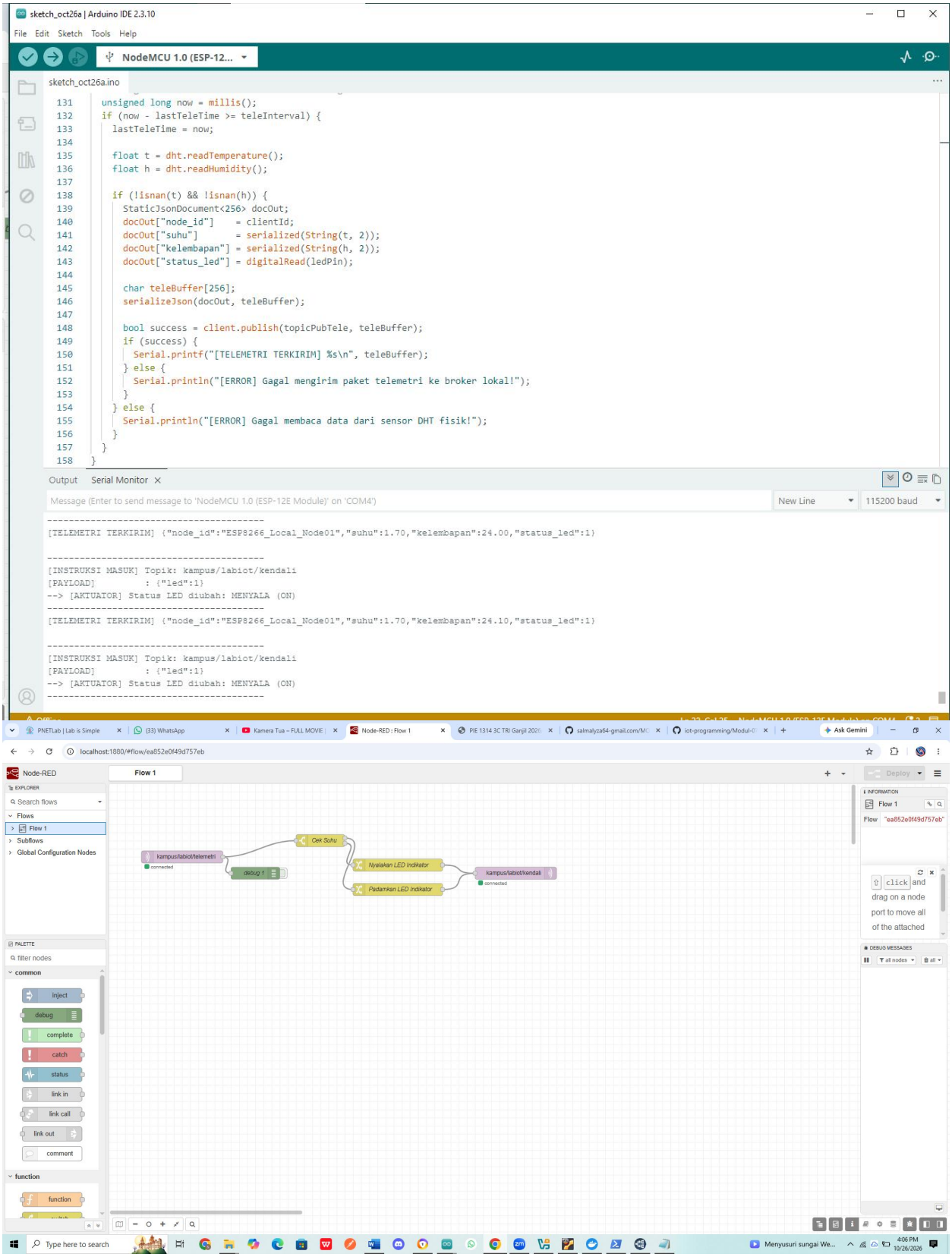

Berdasarkan struktur direktori lokal pada folder iot-server/mosquitto/config/, kendala kegagalan autentikasi awal teratasi melalui penyesuaian berkas sandi (password.txt).

- **Pembersihan Karakter Tersembunyi (*Encoding / Invisible Character Clean-up*):** Saat file teks dibuat atau disunting melalui aplikasi teks editor di Windows, terkadang sistem menyisipkan *Byte Order Mark* (BOM) atau format baris tersembunyi (seperti CRLF) di dalam baris datanya. Karakter tak kasat mata ini membuat fungsi hashing enkripsi Mosquitto gagal membaca string sandi akun admin dengan benar. Dengan membuat file teks baru secara bersih, format data di dalamnya menjadi murni dan langsung bisa dikenali oleh broker.

- **Penyegaran Cache Pembacaan Berkas pada Docker Volume (*File Refresh & Bind Mount Synchronization*):** Kontainer Mosquitto yang sedang berjalan seringkali menahan atau melakukan *cache* terhadap status pembacaan file konfigurasi sandi yang sebelumnya sempat mengalami error *Unauthorized*. Dengan mengubah nama file lama menjadi old.txt dan membuat file baru beranamakan persis password.txt sesuai parameter password_file di dalam mosquitto.conf, kontainer mendeteksi adanya pembaruan atau pergantian objek file secara mutlak pada direktori *mount*-nya.

- **Resolusi Akses Berkas (*File Permission & Release Lock*):** Tindakan mengganti nama file (penghapusan pointer file lama dan pembuatan pointer file baru) memaksa sistem operasi dan kontainer untuk melepaskan *lock* akses berkas yang sebelumnya bermasalah. Begitu file baru terbaca dengan struktur yang valid setelah kontainer di-restart, server Mosquitto kembali membuka izin otorisasi, sehingga klien (ESP8266 dan Node-RED) yang mengirimkan kredensial admin / rahasia123 langsung diterima kembali dengan status sukses berkaliber **rc=0**.




sketch_oct26a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct26a.ino
unsigned long now = millis();
if (now - lastTeleTime >= teleInterval) {
lastTeleTime = now;
float t = dht.readTemperature();
float h = dht.readHumidity();
if (!isnan(t) && !isnan(h)) {
StaticJsonDocument<256> docOut;
docOut["node_id"] = clientId;
docOut["suhu"] = serialized(String(t, 2));
docOut["kelembapan"] = serialized(String(h, 2));
docOut["status_led"] = digitalRead(ledPin);
char teleBuffer[256];
serializeJson(docOut, teleBuffer);
bool success = client.publish(topicPubTele, teleBuffer);
if (success) {
Serial.printf("[TELEMETRI TERKIRIM] %s\n", teleBuffer);
} else {
Serial.println("[ERROR] Gagal mengirim paket telemetri ke broker lokal!");
}
} else {
Serial.println("[ERROR] Gagal membaca data dari sensor DHT fisik!");
}
}
}
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
New Line
115200 baud
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.70,"kelembapan":24.00,"status_led":1}
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD] : {"led":1}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.70,"kelembapan":24.10,"status_led":1}
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD] : {"led":1}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)
localhost:1880/#flow/ea852e0f49d757eb
Node-RED
Flow 1
Search flows
Flows
Flow 1
Subflows
Global Configuration Nodes
kampus/labiot/telemetri
connected
debug 1
Cek Suhu
Nyalakan LED indikator
Padamkan LED indikator
kampus/labiot/kendali
connected
filter nodes
common
inject
debug
complete
catch
status
link in
link call
link out
comment
function
function
Deploy
Flow 1
Flow "ea852e0f49d757eb"
click and drag on a node port to move all of the attached
DEBUG MESSAGES
all nodes
Type here to search
4:06 PM
10/26/2026

Kode:

```
/*
 *
==========
 * PRAKTIKUM 7: SIMPUL SENSOR-AKTUATOR IOT UNTUK BROKER LOKAL
 *
==========
 * Program ini membaca sensor DHT11/22, menyusun paket telemetri JSON, mengirimkannya
 * ke Broker Mosquitto Lokal di Laptop, serta mendengarkan instruksi kendali LED.
 *
==========
 */

#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid     = "salma";
const char* password = "0987654321";

// 2. Konfigurasi Broker MQTT Lokal
```

```
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
const char* mqttServer = "10.171.247.24";
const int   mqttPort   = 1883;

// 3. Konfigurasi Identitas Simpul & Topik
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

// 4. Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)

#define DHTTYPE DHT11 // Ganti DHT22 jika modul sensor berwarna putih

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTeleTime = 0;
const unsigned long teleInterval = 4000; // Siklus kirim data setiap 4 detik

// Penanganan Paket Masuk dari Node-RED melalui Broker Lokal
```

```
void onMessageCallback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("\n----------------------------------------");
  Serial.printf("[INSTRUKSI MASUK] Topik: %s\n", topic);
  Serial.printf("[PAYLOAD]         : %s\n", incomingBuffer);

  // Parsing JSON Instruksi Kendali: {"led": 1} atau {"led": 0}
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);

  if (err) {
    Serial.printf("[ERROR] Gagal parsing JSON instruksi: %s\n", err.c_str());
    return;
  }

  // Evaluasi Kendali Indikator LED
  if (doc.containsKey("led")) {
    int stateLED = doc["led"];
    digitalWrite(ledPin, stateLED == 1 ? HIGH : LOW);
    Serial.printf("--> [AKTUATOR] Status LED diubah: %s\n", stateLED == 1 ? "MENYALA
(ON)" : "PADAM (OFF)");
```

```
  } else {
    Serial.println("[WARNING] Kunci 'led' tidak ditemukan dalam payload!");
  }
  Serial.println("-----------------------------------------");
}

void connectToWiFi() {
  Serial.println();
  Serial.print("[Wi-Fi] Menghubungkan ke ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n[Wi-Fi] Berhasil Terhubung!");
  Serial.print("[Wi-Fi] IP NodeMCU: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
```

```
  while (!client.connected()) {
    Serial.print("[MQTT] Menghubungkan ke Broker Lokal (");
    Serial.print(mqttServer);
    Serial.print(")...");

    if (client.connect(clientId, "admin", "rahasia123")) {
      Serial.println(" Terhubung!");
      // Segera daftarkan subskripsi topik kendali
      client.subscribe(topicSubCmd);
      Serial.printf("[MQTT] Berlangganan ke topik: %s\n", topicSubCmd);
    } else {
      Serial.printf(" Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Awal sistem: LED padam
```

```
  dht.begin();
  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
  client.setCallback(onMessageCallback);
  client.setBufferSize(512);
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectToWiFi();
  }

  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop(); // Mendengarkan instruksi masuk

  // Pengiriman Telemetri Periodik Non-Blocking
  unsigned long now = millis();
  if (now - lastTeleTime >= teleInterval) {
    lastTeleTime = now;
```

```
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (!isnan(t) && !isnan(h)) {
    StaticJsonDocument<256> docOut;
    docOut["node_id"]    = clientId;
    docOut["suhu"]       = serialized(String(t, 2));
    docOut["kelembapan"] = serialized(String(h, 2));
    docOut["status_led"] = digitalRead(ledPin);

    char teleBuffer[256];
    serializeJson(docOut, teleBuffer);

    bool success = client.publish(topicPubTele, teleBuffer);
    if (success) {
      Serial.printf("[TELEMETRI TERKIRIM] %s\n", teleBuffer);
    } else {
      Serial.println("[ERROR] Gagal mengirim paket telemetri ke broker lokal!");
    }
  } else {
    Serial.println("[ERROR] Gagal membaca data dari sensor DHT fisik!");
  }
}
```

}